2.10. Ejecución de procedimientos

- Los procedimientos almacenados en la BD consiste en un conjunto de sentencias SQL y del lenguaje procedural que utiliza el SGBD para las tareas sobre la BD.
- Pueden definirse con parámetros de entrada (IN), salida (OUT), entrada/salida (IN/OUT) o sin parámetros.

```
CREATE PROCEDURE subida_sal (d INT, subida INT)
BEGIN
 UPDATE empelados SET salario = salario + subida WHERE dept_no=d;
 COMMIT;
END;
```

```
CREATE OR REPLACE FUNCTION nombre_dep (d NUMBER, locali OUT
          varchar2)  return varchar2 as nom VARCHAR2(15);
BEGIN
  SELECT dnombre, loc, INTO nom, locali FROM departamentos WHERE
          dept_no=d; return nom;
EXCEPTION
  WHERE NO_DATA_FOUND THEN  nom:= 'INEXISTENTE';
  RETURN nom;
END
```

2.10. Ejecución de procedimientos

La interfaz **CallableStatement** permite que se pueda llamar desde Java a los procedimientos almacenados. Para crear un objeto se llama al método *prepareCall(String)* delobjeto *Connection*.

Ejemplo. Declaración de la llamada al procedimiento *subida_sal* que tiene dos parámetros y se les da valor con el marcador de posición **?**.

```
String sql = "{call subida_sal(?,?)}";
CallableStatement llamada=conexion.prepareCall(sql);
```

Cuatro formas de declarar llamadas a los procedimientos y funciones dependiendo de si hay o no parámetros y de la devolución de valores.

- ✓ *[call procedimiento}* para un procedimiento almacenado sin parámetros.
- ✓ *{?=call function}* para una función almacenada que devuelve un valor y no recibe parámetros. El valor se recibe a la izquierda del igual y es el primer parámetro.
- ✓ *{call procedimiento(?, ?, ...)}* para un procedimiento almacenado que recibe parámetros.
- ✓ *{call function(?, ?, ...)}* para una función almacenada que devuelve un valor (primer parámetro) y recibe varios parámetros.

2.11. Gestión de errores

Hasta ahora, cuando se producía un error la secuencia de llamadas al método que ha producido el la excepción y la línea de código donde se producía el error se visualizaba con *printStackTrace()*.

Cuando se produce un error con **SQLException** podemos acceder a cierta información usando los siguientes métodos.

- *getMessage().* Devuelve una cadena que describe el error.
- *getSQLState().* Es una cadena que contiene un estado definido por el estándar X/OPEN SQL.
- *getErrorCode().* Es un entero que proporciona el código de error del fabricante.

```
try
  { // cídigo }
catch (ClassNotFoundException cn) {cn.printStackTrace));}
catch {SQLException e)
{
  System.out.println("HA OCURRIDO UNA EXCEPCIÓN:");
  System.out.println("Mensaje   :"+e.getMessage());
  System.out.println("SQL estado:"+e.getSQLState());
  System.out.println("Cod error :"+e.getErrorCode());
}
```